

Name Student 1: **Andrea Sabbatini**

Name Student 2: **Adham Hamdy**

Name Student 3: **Irina Alexandra Ciurescu**

1.1 Simple CNN

- Report the sizes of the training and validation sets downloaded from the MNIST dataset

	Image height	Image width	# image channels	# samples
Training	28 pixels	28 pixels	1	60000
Validation	28 pixels	28 pixels	1	10000

- Report the number of parameters of the SimpleCNN model

	# trainable parameters
Simple CNN	813802

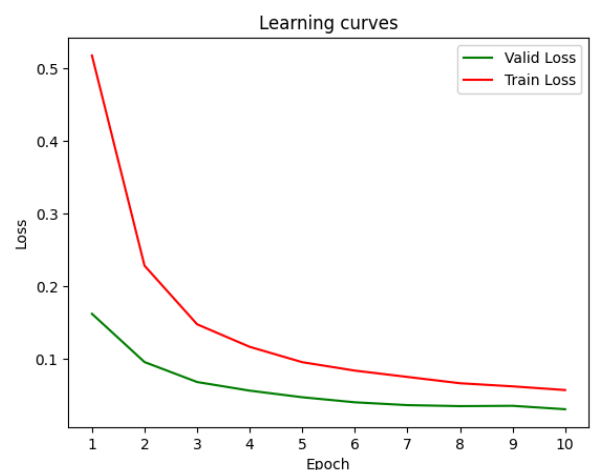
- Include the training and validation curves for 10 epochs. Also indicate the best accuracy result and in which epoch is this achieved
Comment on the conclusions of the training and validation losses, regarding possible i) high-bias or ii) overfitting problems. Indicate if you consider further training would improve the performance of the model.

	Best accuracy (validation)	Epoch with best accuracy
Simple CNN	99.01%	10

We can see that there is no indication of high bias, since we don't have high training loss, low training accuracy nor poor validation performance. Instead, validation accuracy reaches 99% and training loss becomes very low.

Moreover, regarding overfitting, we can see that there is no divergence between train and validation curves. Validation loss is even lower than training loss and they both keep decreasing smoothly. So there are no signs of overfitting.

Since we have very high validation accuracy and low validation loss (0.03), if we train more epochs we think it will introduce overfitting problems, while bringing very small improvements in accuracy.



- Include the confusion matrix. Given this confusion matrix, report which are the 2 cases of confusion between classes that happen more frequently, and mark over the matrix which values you are using to identify those cases. By inspection of the mistake examples in the notebook, attach examples of the confusion cases you have identified.

true label: 7 (0.008)
pred label: 2 (0.992)

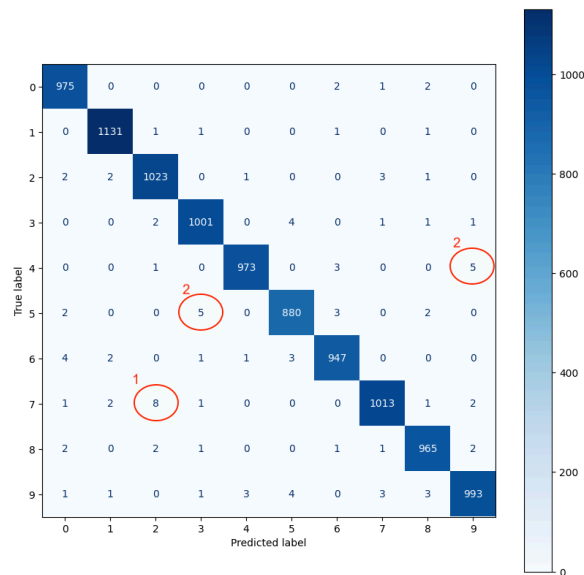


true label: 5 (0.100)
pred label: 3 (0.900)



true label: 4 (0.019)
pred label: 9 (0.980)



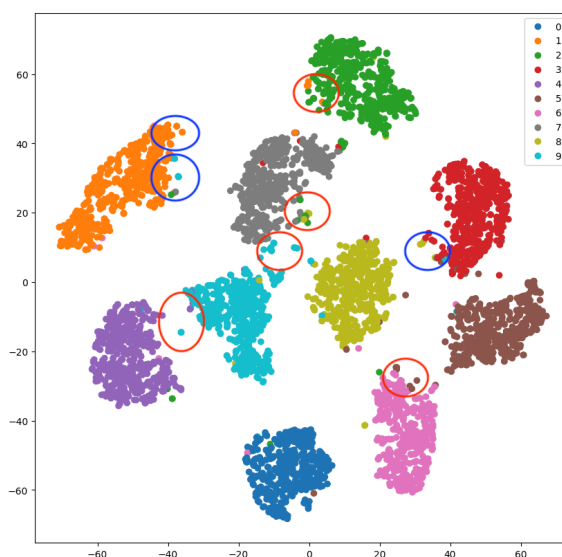


We can see that the model confused 8 times the 7 (true label) with the 2 (predicted label). This is still a very low value, since the model got 1013 times the 7 (true label) right.

The second most seen error is represented by confusing the 5 (true label) with the 3 (predicted label) or the 4 (true label) with the 9 (predicted label). These confusions both happened 5 times, while the model got right the 5 (true label) 880 times and the 4 (true label) 973 times. The number of confusions is still pretty low. We can see in the examples of confusions shown that the errors are quite understandable even for a human eye.

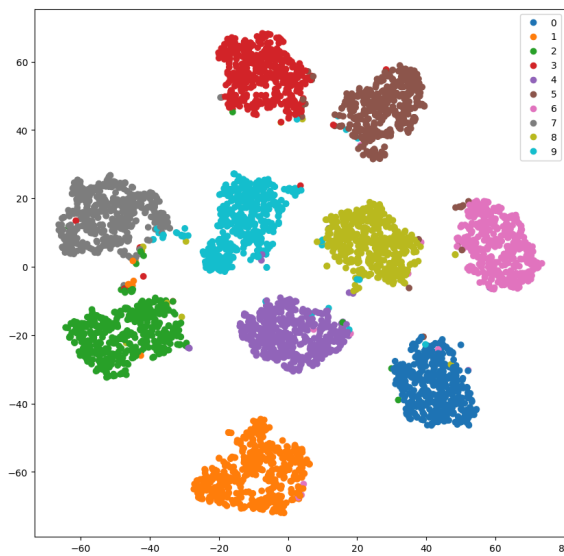
- Comment on the differences between the t-SNE graph of the representation of the final and intermediate layers of the CNN, applied to the images of the validation set. In your analysis:
 - o Mark in the intermediate-layer graph which data should correspond with prediction error cases
 - o Consider the proximity and the dispersion of the clusters in both representations to derive conclusions on the capacity of the model to separate image classes at both network layers.

This is the intermediate-layer graph:



We need to check for predictions that are found between different clusters, that are far from the belonging cluster's center, that are in a different one or that are at cluster boundaries: these correspond with prediction error cases. We can see that the clusters are not fully structured, and the predictions easily diverge from the center (the model is not quite sure about its answers). For example, the 1's cluster is very sparse and there is still a lot of visible confusion (2 and 7, 1 and 2, 7 and 9).

This is the final-layer graph:



We can see that clusters are very compact and there is a clear separation between the classes. Now the model is more sure about its decisions, for example we can see that the 1's cluster is more compact, meaning that it's identified with more certainty. Some numbers are recognized as more different than before, since the distance between them increased (for example, 5 and 6, 1 and 2). Some confusion is still visible, like the one between 7, 2 and 9.. We can see it because the clusters are still near and some identifications are outside the right clusters. This is coherent with the considerations done on the confusion matrix.

- Given the differences between the t-SNE representation of both layers, and using the architecture definition: 1) identify which network layers correspond to each extracted representation (intermediate and final), and 2) propose a way to reduce the model size with only a low penalty in classification accuracy.

The intermediate representation represents the situation after the first fully connected layer, with 128 features units. The final representation, instead, represents the situation after the second fully connected layer, with 10 features.

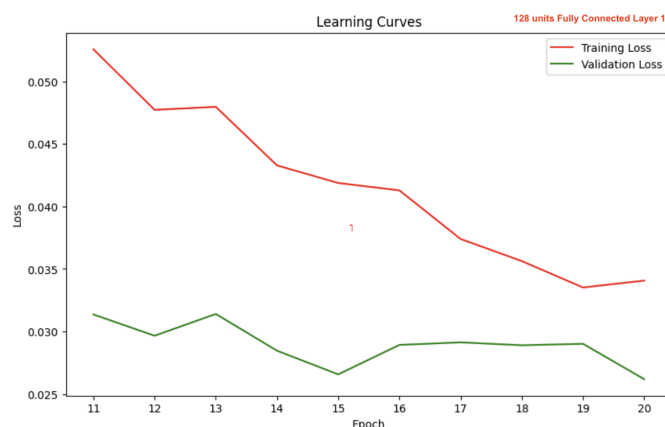
Regarding a way to reduce the model size without losing too much accuracy, we came up with and tested two possible solutions:

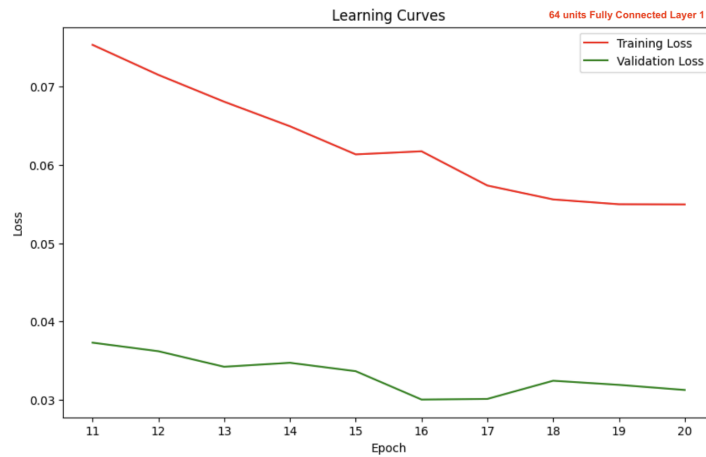
- Reducing the number of units in the first fully connected layer from 128 to 64.

This operation will halve the number of learnable parameters in the first fully connected part, which contributes the most to the total number of learnable parameters.

[001]	train loss: 0.612390	val loss: 0.181116	val acc: 94.6500%
[002]	train loss: 0.261065	val loss: 0.086210	val acc: 97.3500%
[003]	train loss: 0.168873	val loss: 0.063109	val acc: 98.1300%
[004]	train loss: 0.140071	val loss: 0.055407	val acc: 98.1700%
[005]	train loss: 0.115670	val loss: 0.046705	val acc: 98.4700%
[006]	train loss: 0.108025	val loss: 0.042856	val acc: 98.5900%
[007]	train loss: 0.094920	val loss: 0.043756	val acc: 98.5300%
[008]	train loss: 0.089239	val loss: 0.039619	val acc: 98.7500%
[009]	train loss: 0.082329	val loss: 0.036706	val acc: 98.8000%
[010]	train loss: 0.080615	val loss: 0.035559	val acc: 98.7100%

We can see that in the first 10 epochs the values are very similar to before, with only a small gap. If we continue training for 10 more epochs, we can see that the model overfits more than before.





2) Concatenate two 2x2 max pooling

If we add another 2x2 max pooling before the first fully connected layer, the number of input features in the FC layer is reduced by 75% (from $32 \times 14 \times 14$ to $32 \times 7 \times 7$). This means that the number of learnable parameters in the first fully connected layer, which is the one that contributes the most to the total number of learnable parameters, is reduced too by (a bit less than) 75%.

We can see that the accuracy improves even with respect to the original SimpleCNN, reaching 99.03% in the first ten epochs, while the losses slightly increase.

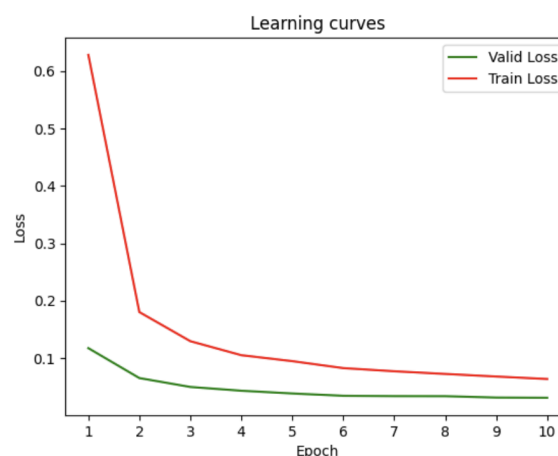
Two 2x2 max-pooling

```
[001] train loss: 0.627942 val loss: 0.117324 val acc: 96.4700%
[002] train loss: 0.180186 val loss: 0.065279 val acc: 97.9000%
[003] train loss: 0.129485 val loss: 0.049784 val acc: 98.4300%
[004] train loss: 0.105103 val loss: 0.043221 val acc: 98.5800%
[005] train loss: 0.094738 val loss: 0.038452 val acc: 98.7600%
[006] train loss: 0.082612 val loss: 0.034432 val acc: 98.8800%
[007] train loss: 0.077116 val loss: 0.033863 val acc: 98.8600%
[008] train loss: 0.072466 val loss: 0.033748 val acc: 98.9200%
[009] train loss: 0.067950 val loss: 0.031307 val acc: 98.9900%
[010] train loss: 0.063673 val loss: 0.030972 val acc: 99.0300%
```

One 2x2 max-pooling (original SimpleCNN)

```
[001] train loss: 0.517658 val loss: 0.161693 val acc: 95.1500%
[002] train loss: 0.227977 val loss: 0.095162 val acc: 97.1100%
[003] train loss: 0.147224 val loss: 0.067661 val acc: 97.7600%
[004] train loss: 0.116178 val loss: 0.055791 val acc: 98.2700%
[005] train loss: 0.095021 val loss: 0.046678 val acc: 98.4600%
[006] train loss: 0.083410 val loss: 0.039715 val acc: 98.7700%
[007] train loss: 0.074683 val loss: 0.035823 val acc: 98.8200%
[008] train loss: 0.065983 val loss: 0.034452 val acc: 98.8400%
[009] train loss: 0.061696 val loss: 0.034870 val acc: 98.8100%
[010] train loss: 0.056613 val loss: 0.030176 val acc: 99.0100%
```

This solution simplifies the model by reducing the resolution of the features, which brings the model to be less sensible and way faster in learning, how we can see in the graph:



We conclude that, since the number of parameters is reduced by a lot and the performance essentially improves, this solution is, in general, better than the original SimpleCNN.

1.2 AlexNet

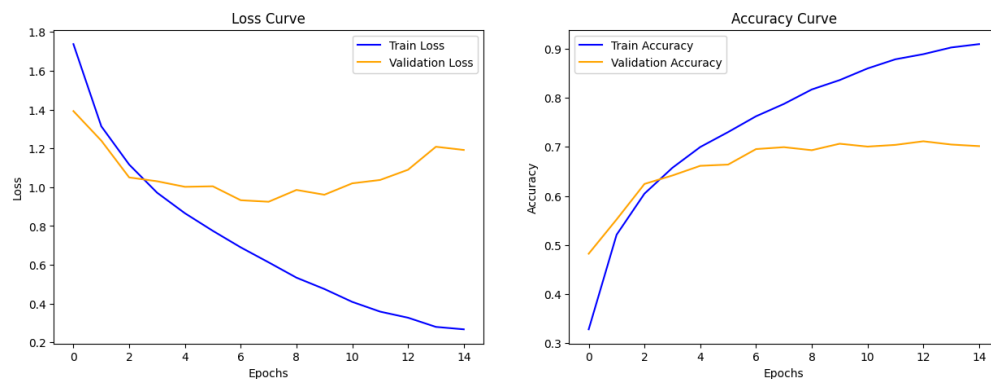
- Report the number of parameters of the AlexNet model

	# trainable parameters
AlexNet	6199498

- Include the training and validation curves for 15 epochs. Also indicate the best accuracy result and in which epoch is this achieved

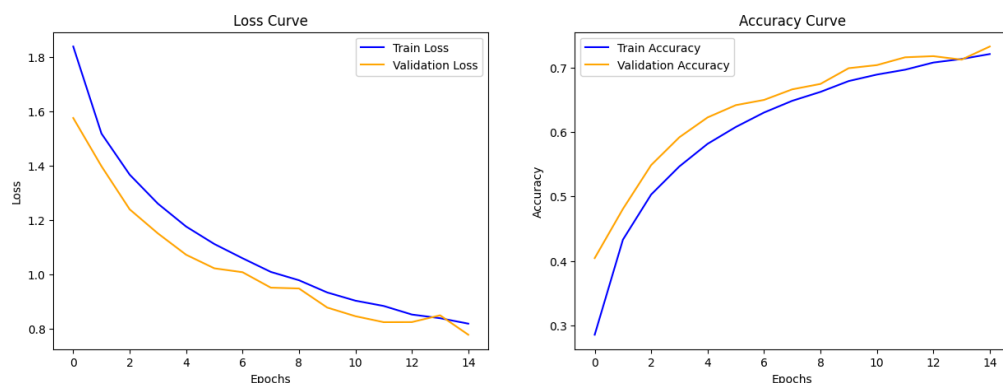
	Best accuracy (validation)	Epoch with best accuracy
AlexNet	71.13%	13

Comment on the conclusions of the training and validation losses, and remark what is possibly happening after epoch 10. Indicate if you consider further training would improve the performance of the model



We can see that (without augmentation) the model is overfitting after the 6/7 epoch. In fact, the gap between validation loss/accuracy and training loss/accuracy increases. For this reason, we don't think that further training would improve the performance, also because we can see that, after the 10th epoch, validation loss is increasing while the accuracy stays more or less the same. Probably after the 10th epoch, as a consequence of overfitting, the model is being so confident on the wrong guesses (which makes the loss explode).

Instead, we tried to implement some augmentation on the dataset, specifically horizontal flipping and random crop. We can see that now it's harder for the model to learn (look at train accuracy/loss), but the model gains validation accuracy from training for more epochs. From now on, we will consider the data gained by training with augmentation.



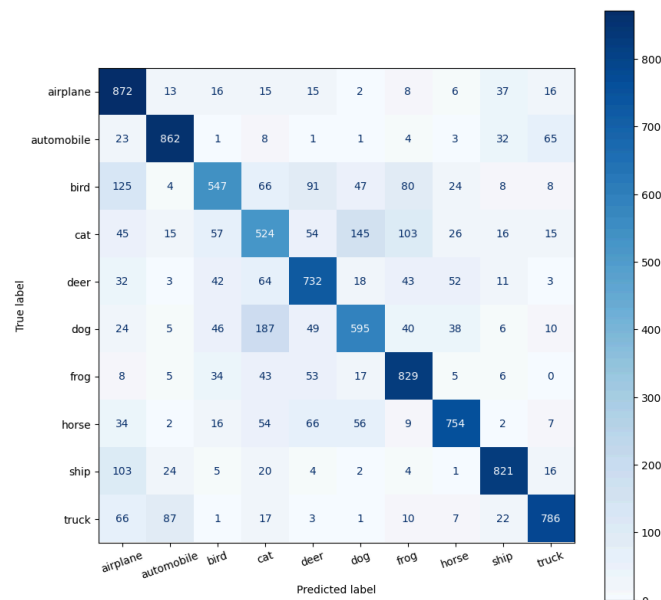
- Include here the per-class accuracy results (you have an empty code block in the notebook prepared for that purpose)

```

Accuracy for airplane: 87.20%
Accuracy for automobile: 86.20%
Accuracy for bird: 54.70%
Accuracy for cat: 52.40%
Accuracy for deer: 73.20%
Accuracy for dog: 59.50%
Accuracy for frog: 82.90%
Accuracy for horse: 75.40%
Accuracy for ship: 82.10%
Accuracy for truck: 78.60%

```

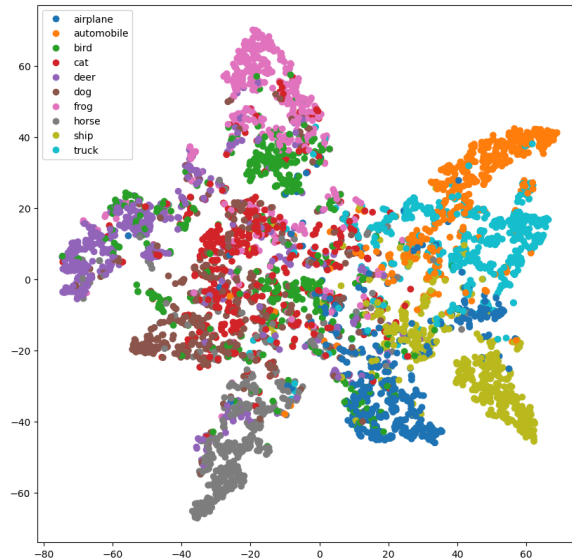
- Include the confusion matrix. Comment on the confusion matrix data attending to the visual characteristics of the image classes. In your response, do not merely indicate the most common failure cases. In your analysis:
 - o Comment on the possible existence of groups of categories: i.e. several categories that are commonly confused between each other
 - o The visual characteristics of those category groups
 - o Establish your hypothesis on the explanation behind the distribution of per-class accuracies once you have seen the confusion matrix results



We can see that the model recognizes some groups of categories. For example, he confuses a lot the airplane, bird and ship classes. This visually makes sense because they are often in the same environment (blue background) and, sometimes, they have similar shapes. It's interesting to see that the model, if it's "confused", prefers to guess airplane, especially with birds (look at the accuracy for airplane and the accuracy for bird).

Another group of categories contains the quadruped animals (cat, dog, horse, deer). The model here doesn't prefer a particular class (when it's "confused"), and we can see it from the accuracies per class: they have really low accuracy compared to others (especially cat and dog: it looks like it chooses randomly when it faces one of them)

- Include the t-SNE results for the final layer: discuss on these results (proximity, dispersion, cluster aggrupation) considering the visual appearance of the different classes: their typical visual features, and compare the results with the t-SNE results in the MNIST dataset.



The t-SNE visualization plot for CIFAR-10 shows more diffuse and overlapping clusters than the one for MNIST. In general, it looks like classes generally overlap near the center, with some diverged clusters that represent the best recognized classes (look at frog, automobile, ship). We can see that our interpretation of the cat and dog per class accuracy is quite right: the cat and dog clusters are essentially in the same feature space, and they aren't really grouped. This means that, for the model, they have quite the same characteristics.

MNIST shows a clear structure of handwritten digits, while CIFAR-10 reflects higher intra-class variability. The images here come from natural scenes, background and lightning differ. This makes the learned feature representations less clearly separated in the embedding space.

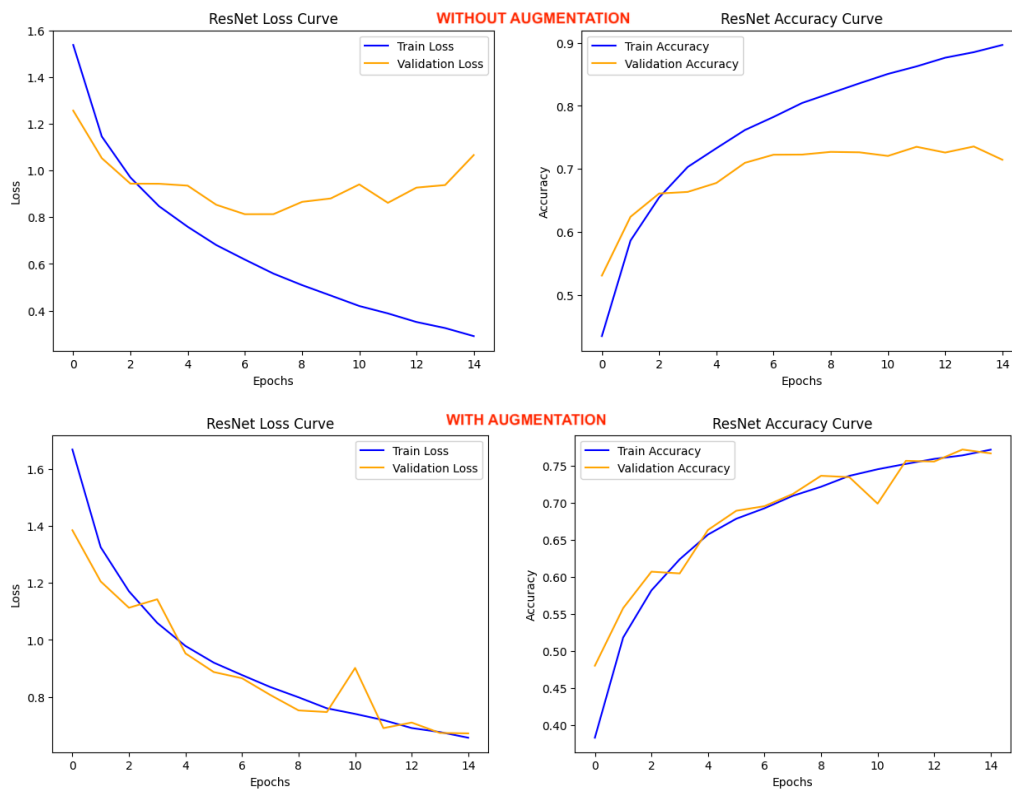
1.3 Basic Residual Network

- Report the number of parameters of the BasicResNet model

	# trainable parameters
BasicResNet	151690

- Include the training and validation curves for 15 epochs. Also indicate the best accuracy result and in which epoch is this achieved

	Best accuracy (validation)	Epoch with best accuracy
BasicResNet	73.55%	14



- Comment on the trade-off between computational complexity (in terms of model size) and accuracy in the implemented AlexNet and ResNet models. Compare this trade-off in both architectures to achieve a conclusion on the efficiency of these two different architectures.

AlexNet has a larger parameter count and it is dominated mainly by the fully-connected part, which is expensive and can be the cause of the overfitting. BasicResNet has a much smaller footprint and lower memory usage, which makes the training faster.

We tried to train ResNet normally, but also with data augmentation, like we did for AlexNet. With data augmentation, like it happened before, the overfitting tendency decreases and validation accuracy improves, reaching around 77.23% and the training accuracy is 76.44%.

Now, to compare the two architectures, BasicResNet reaches higher validation accuracy both with and without augmentation, despite being dramatically smaller. Residual connections help gradient flow and allow deeper feature learning without collapsing optimization. This tends to produce features that generalize better on natural images. AlexNet's higher complexity can push training accuracy up, but without strong regularization or augmentation, it will just overfit and not increase the validation as well.

Thus, BasicResNet is more efficient. It achieves better validation accuracy with less parameters than AlexNet, providing a clear better accuracy-to-model-size trade-off for this task.